ADR_001 模組化單體與 Spring Modulith

> 狀態：已接受
> 日期：2026-10-14

背景

平台有 13 個業務模組（《設計_001_領域模型》第 4 節）。專案目標包含分散式系統，但一開始就拆成微服務，會在業務流程還沒做對之前，就先背上服務間通訊、分散式交易、部署與除錯的成本。

同時，模組之間的界線必須從第一天就清楚，否則日後拆分服務時會牽一髮動全身。

考慮過的選項

選項	優點	缺點
一開始就拆成微服務	直接面對分散式問題	業務還沒穩定就先付出大量基礎設施成本；交易邊界一開始就很難做對
傳統單體，不刻意劃分模組	開發最快	模組之間很快互相依賴，日後無法拆分
模組化單體，以慣例約束	單一部署，界線清楚	慣例只靠人遵守，時間久了必然被打破
模組化單體，以 Spring Modulith 驗證	單一部署；模組邊界由測試自動檢查；內建事件持久化	依賴特定框架的套件結構慣例

決定

採用模組化單體，以 Spring Modulith 組織與驗證模組：

- 每個模組是一個頂層套件，對外只公開頂層套件中的介面與事件，實作放在 `internal` 子套件
- 以 Spring Modulith 的驗證測試檢查模組之間沒有違規依賴與循環依賴，違反時建置失敗
- 模組之間的事件使用 Spring Modulith 的事件持久化機制（見 ADR_007）
- 由 Spring Modulith 產生模組關係文件，與《設計_001_領域模型》對照

後果

- 好處：第一版只需部署一個應用程式，可以專注在業務正確性；模組邊界有自動化保證，日後拆分服務時範圍明確
- 代價：所有模組共用一個資料庫與一個行程，單一模組的效能問題會影響整體，這正是之後壓測要觀察的
- 重新檢討的條件：某個模組的負載特性明顯不同（例如搶購）而需要獨立擴展，或需要獨立部署時，拆出該模組並寫新的 ADR